

ECOM EXPERTS

Juegos

Manual de usuario, técnico y de desarrollo

juegos · 08/09/2026

Contenido

"Ánima" — Juego 2D para PC (Windows) y Android con publicidad AdMob

Spec de ejecución para Antigravity + Claude — Windows 11

Versión 1.0 — julio 2026

0. RESUMEN EJECUTIVO

los sprites, genera tu si puedes o dame los prompt para añadirlos a otra IA

1. INSTRUCCIONES PARA ANTIGRAVITY (LEER PRIMERO)

2. HERRAMIENTAS (qué instala el bootstrap y para qué)

3. ARQUITECTURA DEL PROYECTO GODOT

4. FASES DE DESARROLLO

FASE 1 — Proyecto base y bucle mínimo

FASE 2 — Objetos

FASE 3 — Enemigos: bolas de energía

FASE 4 — Generación procedural de niveles ★

FASE 5 — Dificultad, tiempo y progresión

FASE 6 — Sprites y animaciones por código ★

FASE 7 — Audio

FASE 8 — Export Windows

FASE 9 — Export Android

FASE 10 — Publicidad AdMob ★

FASE 11 — Alta en Google Play (estado a julio 2026)

FASE 12 — Alta en AdMob

5. CRITERIOS DE "HECHO" (auditoría final antes de release)

"Ánima" — Juego 2D para PC (Windows) y Android con publicidad AdMob

Spec de ejecución para Antigravity + Claude — Windows 11

Versión 1.0 — julio 2026

0. RESUMEN EJECUTIVO

Juego 2D top-down de pantalla única (no laberinto, arena abierta con scroll opcional):

- **Protagonista:** un fantasma que se desplaza libremente por la pantalla.
- **Objetos:** repartidos por el nivel. Tipos: dan puntos, matan, curan, otorgan propiedades (velocidad, invisibilidad, escudo, imán de maná...).
- **Enemigos:** bolas de energía que persiguen al fantasma; al tocarlo restan vida o matan.
- **Objetivo por nivel:** encontrar la salida O acumular suficiente **maná** antes de que acabe el **tiempo límite**.
- **Progresión:** cada nivel es más difícil — más enemigos, más rápidos, nuevos tipos de objeto, nuevos peligros, menos tiempo.
- **Niveles:** generados **proceduralmente por código** (sin editor manual de mapas).
- **Sprites y animaciones:** generados **por código** (pipeline Python/PIL → spritesheets PNG), sin depender de artista.
- **Monetización:** AdMob en Android (banner + interstitial + rewarded). La versión PC no lleva anuncios (AdMob no funciona en escritorio; la versión Windows se distribuye gratis en itch.io o de pago).

Stack: Godot 4.7 stable + GDScript · JDK 17 · Android SDK (API 35/36) · Python 3 + Pillow para sprites · plugin AdMob de Poing Studios.

los sprites, genera tu si puedes o dame los prompt para añadirlos a otra IA

1. INSTRUCCIONES PARA ANTIGRAVITY (LEER PRIMERO)

1. Ejecutar `bootstrap_entorno_juego.ps1` en PowerShell **como Administrador**. No requiere reiniciar Windows. Verificar al final que todos los checks salen `[OK]`.
2. Trabajar las fases en orden. **No pasar de fase sin que la anterior compile y ejecute.**
3. Rutas base: - Herramientas: `C:\dev\` - Proyecto: `C:\dev\anima\` (proyecto Godot) - Pipeline de sprites: `C:\dev\anima\tools\`
4. Godot se ejecuta en headless para tests: `godot --headless --path C:\dev\anima --quit` (valida que el proyecto carga sin errores de parseo).

5. Antes de declarar una fase terminada: ejecutar el juego, capturar la salida de consola, comprobar que no hay errores ni warnings de script.
6. Todo el código GDScript con `class_name` donde aplique, tipado estático (`var vida: int = 100`), y señales en vez de acoplamiento directo.

2. HERRAMIENTAS (qué instala el bootstrap y para qué)

Herramienta	Uso	Instalación
Godot 4.7 stable (win64)	Motor del juego, editor y export	winget / descarga directa
Export templates 4.7	Plantillas para exportar a Windows y Android	descarga + colocación en <code>%APPDATA%\Godot</code>
JDK 17 (Temurin)	Requerido por el export Android de Godot (Gradle)	winget
Android cmdline-tools + SDK	platform-tools, build-tools 35, platforms android-35/36	descarga + <code>sdkmanager</code>
Python 3.12 + Pillow	Generación procedural de spritesheets	winget + pip
Git	Control de versiones del proyecto	winget
ImageMagick	Utilidades de imagen (montaje, reescalado, atlas)	winget
Krita (opcional)	Retoque manual de sprites si hace falta	winget
Audacity (opcional)	Retoque de sonidos	winget

Sonidos: generarlos por código con el script `tools/gen_sfx.py` (síntesis simple estilo sfxr con `numpy` + `wave`) o con jsfxr web. No comprar assets.

3. ARQUITECTURA DEL PROYECTO GODOT

```
C:\dev\anima\
├─ project.godot
├─ export_presets.cfg
├─ scenes/
│   ├── main.tscn          # gestor de partida (Game)
│   ├── player.tscn        # fantasma
│   ├── enemy_orb.tscn     # bola de energía base
│   ├── item.tscn          # objeto genérico parametrizable
│   ├── exit_portal.tscn   # salida del nivel
│   ├── hud.tscn           # vida, maná, tiempo, puntos
│   └─ menus/ (title, game_over, pause, level_clear)
├─ scripts/
│   ├── game.gd            # máquina de estados de partida
│   ├── player.gd
│   ├── enemy_orb.gd
│   ├── item.gd
│   ├── level_generator.gd # ★ generación procedural
│   ├── difficulty.gd      # ★ curva de dificultad
│   ├── item_db.gd         # catálogo de objetos (Resource)
│   ├── save.gd            # récords, nivel máximo, opciones
│   └─ ads_manager.gd      # ★ AdMob (autoload, no-op en PC)
├─ assets/
│   ├── sprites/          # PNG generados por el pipeline
│   ├── sfx/
│   └─ fonts/
├─ addons/
│   └─ admob/              # plugin Poing Studios
└─ tools/
    ├── gen_sprites.py     # ★ pipeline de spritesheets
    └─ gen_sfx.py
```

Autoloads (Project Settings → Autoload): `Difficulty` , `Save` , `AdsManager` .

4. FASES DE DESARROLLO

FASE 1 — Proyecto base y bucle mínimo

- Crear proyecto Godot 4.7, renderer **Mobile** (Vulkan mobile), resolución base 1280×720, stretch `canvas_items` + `expand` .
- Input map: teclado (WASD/flechas) + **joystick virtual táctil** para Android (usar `VirtualJoystick` nativo de Godot 4.7 o `TouchScreenButton`).
- `player.tscn` maná (recurso para subir de nivel).
- `CURA` — restaura vida.
- `LETAL` — mata o hace daño grave al tocarlo (calaveras, trampas).

- `POWERUP` — propiedad temporal: `velocin : CharacterBody2D + AnimatedSprite2D + CollisionShape2D`. Movimiento con aceleración/fricción (el fantasma "flota": inercia suave, `lerp` de velocidad).
- Escena main con fondo placeholder y el fantasma moviéndose.
- ✓ Test: se mueve suave con teclado y con ratón/touch (opcional: moverse hacia donde se pulsa).

FASE 2 — Objetos

- `item.tscn : Area2D` parametrizable por `ItemData` (Resource) con campos: `id`, `nombre`, `tipo`, `valor`, `sprite_frame`, `rareza`, `nivel_minimo`.
- Tipos (`enum ItemType`):
- `PUNTOS` — suma puntuación.
- `MANA` — suma `dad`, `escudo`, `invisibilidad` (los orbes no te ven), `iman_mana`, `congelar_enemigos`, `vida_extra`.
- `item_db.gd`: catálogo con ~20 objetos, cada uno con `nivel_minimo` → así "salen nuevos objetos" al subir de nivel sin tocar código.
- ✓ Test: recoger cada tipo produce su efecto y una pequeña animación (tween de escala + fade).

FASE 3 — Enemigos: bolas de energía

- `enemy_orb.gd : CharacterBody2D` con máquina de estados: `PATRULLA` (deambula con ruido Perlin) → `PERSECUCION` (si el jugador entra en su radio de detección y no es invisible) → `PATRULLA` (si lo pierde).
- Variantes por nivel (mismo script, distintos parámetros y color):
- **Orbe azul** (nivel 1+): lento, quita 25 vida, knockback.
- **Orbe verde** (3+): rápido, quita 15.
- **Orbe rojo** (5+): mata de un toque, detección corta.
- **Orbe violeta** (8+): se teletransporta cerca del jugador cada X s.
- **Orbe dorado** (12+): dispara proyectiles.
- Cooldown de daño (i-frames de 1 s con parpadeo del fantasma).
- ✓ Test: los orbes patrullan, persiguen, dañan, respetan i-frames y la invisibilidad.

FASE 4 — Generación procedural de niveles ★

`level_generator.gd` — todo por código, con **RNG con semilla** (`seed = nivel * 7919 + run_seed`) para reproducibilidad y depuración:

1. Definir arena rectangular; tamaño crece con el nivel (720p → hasta 2× con cámara que sigue al jugador).
2. Colocar 3–8 **obstáculos decorativos** (rocas/nieblas con colisión) mediante rechazo aleatorio.
3. Distribuir objetos con **Poisson disc sampling** (distancia mínima entre objetos) para que no se apilen: - nº de objetos de puntos/maná: `base + nivel` - nº de objetos letales: `floor(nivel * 0.8)` - powerups: 1–3 según rareza ponderada - Solo objetos con `nivel_minimo <= nivel_actual`.
4. Spawn de orbes: `1 + floor(nivel / 2)`, tipos según nivel, **nunca a menos de 300 px del jugador** al inicio.
5. Salida (`exit_portal`): aparece cerrada; se abre al alcanzar `mana_requerido(nivel)` • siempre visible pero lejana (elegir por diseño: recomendado — portal visible desde el inicio en el extremo opuesto, y el maná

como vía alternativa de completar nivel).

- Validación automática del nivel generado: comprobar que ningún objeto/orbe se solapa con obstáculos (raycast/overlap tras generar; si falla, regenerar posición).

FASE 5 — Dificultad, tiempo y progresión

`difficulty.gd` (autoload) expone funciones puras del nivel N:

```
tiempo_limite(N)    = clamp(90 - N*3, 35, 90)    # segundos
mana_requerido(N)   = 50 + N*15
vel_orbe(N)         = 80 + N*8                  # px/s, con tope
num_orbes(N)        = 1 + floor(N/2)
```

- HUD: barra de vida, barra de maná (con marca del requerido), cronómetro, puntos, nivel.
- Al agotarse el tiempo sin salir/maná → muerte.
- `game.gd`: estados `JUGANDO` / `NIVEL_SUPERADO` / `GAME_OVER` / `PAUSA`. Entre niveles, pantalla de resumen 2 s.
- Guardado (`user://save.cfg`): récord de puntos, nivel máximo alcanzado, volumen, idioma.

FASE 6 — Sprites y animaciones por código ★

`tools/gen_sprites.py` (Python + Pillow). Genera todos los PNG en `assets/sprites/`:

- Fantasma** — spritesheet 8 frames × 4 estados, 64×64 px/frame:
 - `idle`: cuerpo blanco semitransparente (elipse + cola ondulada dibujada con curvas seno, fase por frame), ojos oscuros, oscilación vertical.
 - `move`: igual + cola más estirada e inclinación en la dirección.
 - `hit`: tintado rojo + distorsión.
 - `death`: disolución (frames con opacidad y ruido creciente).
- Variantes de color para powerups: escudo (halo cian), invisible (alpha 0.25), veloz (estela).
- Orbes** — 6 frames de pulso: círculo con gradiente radial + glow (blur) + partículas; una paleta por variante (azul/verde/rojo/violeta/dorado).
- Objetos** — iconos 48×48 generados con primitivas: gema (puntos), frasco (cura), estrella (maná), calavera (letal), rayo/reloj/ojo (powerups). Cada uno con animación de 4 frames de brillo/flotación.
- Portal** — anillo giratorio 8 frames, versión cerrada (gris) y abierta (turquesa).
- Salida: un PNG spritesheet por entidad + un `manifest.json` con frames/fps que Godot lee para construir `SpriteFrames` automáticamente (script editor `build_spriteframes.gd` o construcción en runtime).
- Cambios al tocar objeto**: el efecto visual se resuelve con: (a) frame alternativo del fantasma (variante de color del pipeline), (b) `modulate` + tween, (c) `GPUParticles2D` con textura generada. Documentar en el propio script qué combinación usa cada powerup.

Regenerar arte = volver a ejecutar `python tools\gen_sprites.py`. Determinista (semilla fija) para no ensuciar el repo.

FASE 7 — Audio

- `tools/gen_sfx.py` : síntesis de WAV (numpy): recoger objeto (blip ascendente), daño (ruido corto), muerte (descenso), portal (acorde), powerup (arpeggio), tick de últimos 10 s.
- Música: bucle ambiental simple generado (pads con acordes menores) o silencio + ambiente; no usar música con copyright.

FASE 8 — Export Windows

- Preset "Windows Desktop" en `export_presets.cfg`, embed PCK, icono (generado también por el pipeline).
- `godot --headless --export-release "Windows Desktop" build\windows\anima.exe`
- ✓ Test: el exe corre en limpio.

FASE 9 — Export Android

1. Keystore debug ya la crea el bootstrap. Para release: `keytool -genkey -v -keystore C:\dev\keys\anima-release.keystore -alias anima -keyalg RSA -keysize 2048 -validity 10000` **Guardar contraseña en gestor de contraseñas. Sin esa keystore no se puede actualizar la app jamás.**
2. En Godot: Editor Settings → Export → Android: rutas de JDK 17 y Android SDK (`C:\dev\android-sdk`).
3. Preset Android: package `com.TUDOMINIO.anima` (usar dominio propio real), **target SDK 36** (obligatorio para novedades en Play desde 31/08/2026), min SDK 24, arquitecturas arm64-v8a (+armeabi-v7a opcional), formato **AAB** para Play (APK solo para pruebas locales).
4. Activar **Gradle build** (necesario para el plugin AdMob).
5. `godot --headless --export-release "Android" build\android\anima.aab` - ✓ Test: instalar APK debug en un móvil real vía `adb install`.

FASE 10 — Publicidad AdMob ★

Plugin: **Poing Studios Godot AdMob** (<https://github.com/poing-studios/godot-admob-plugin>) — soporta Godot 4.x, banner/interstitial/rewarded y **UMP** (consentimiento GDPR, imprescindible en España/UE).

1. Instalar plugin en `addons/admob/` + descargar el plugin Android (`.aar`) de la release correspondiente a la versión de Godot y colocarlo según README del plugin.
2. `ads_manager.gd` (autoload): - En PC (`OS.get_name() != "Android"`): todo no-op. - `initialize()` → primero flujo **UMP/consentimiento** (formulario GDPR), después `MobileAds.initialize()`. - `show_banner()` — banner adaptativo inferior, solo en menús (nunca tapando el juego). - `show_interstitial()` — al morir o cada 3 niveles superados, **nunca más de 1 cada 90 s** (frecuencia agresiva = usuarios que desinstalan y posible sanción de AdMob). - `show_rewarded(callback)` — voluntario: "ver anuncio para revivir con media vida" o "+30 s de tiempo". Es el formato que mejor paga y mejor tolera el usuario.
3. **Durante TODO el desarrollo usar los IDs de test de Google** (App ID test: `ca-app-pub-3940256099942544~3347511713` ; unidades de test documentadas en developers.google.com/admob/android/test-ads). Clicar tus propios anuncios reales = baneo de AdMob.
4. AndroidManifest: el plugin inyecta el `com.google.android.gms.ads.APPLICATION_ID` desde su configuración; verificar tras el build de Gradle.

5. Cambiar a IDs reales solo en el build final de producción.

FASE 11 — Alta en Google Play (estado a julio 2026)

1. **Cuenta:** play.google.com/console → registrarse. Cuota única de 25 USD. Verificación de identidad obligatoria (DNI) y, como autónomo, valorar registrarse como **organización** con datos de autónomo/empresa en vez de cuenta personal, porque: - *Las cuentas personales creadas después del 13/11/2023 deben superar una prueba cerrada con mínimo 12 testers reales durante 14 días continuos antes de poder solicitar producción; las cuentas de organización están exentas de este requisito.* - Si vas con cuenta personal: *al solicitar acceso a producción, al menos 12 testers deben haber participado en la prueba cerrada durante al menos los últimos 14 días sin interrupciones. Los testers deben instalar desde el enlace de Play Store (no sideload) y conviene reclutar 14–16 para cubrir bajas.*
2. **Recuperar una cuenta antigua:** entrar en play.google.com/console con el Gmail original. Si está inactiva/cerrada: support.google.com/googleplay/android-developer → "Contact us" → recuperación de cuenta (piden verificar identidad y el correo original). Si la cuenta es anterior a nov-2023 y sigue en pie, **no** le aplica el requisito de los 12 testers — merece mucho la pena recuperarla antes que crear una nueva.
3. **Ficha:** título, descripción corta/larga, icono 512×512, feature graphic 1024×500 (generarlos con el pipeline), 4–8 capturas, política de privacidad **obligatoria** (URL propia; al llevar AdMob la app recopila datos → declarar en "Data safety": IDs de dispositivo/publicidad, compartidos con Google).
4. Cuestionarios: clasificación de contenido (IARC), anuncios = Sí, público objetivo (no marcar niños: AdMob + menores complica todo; declarar 13+).
5. Subir AAB firmado → prueba interna → prueba cerrada (los 12 testers si aplica) → producción.

FASE 12 — Alta en AdMob

1. admob.google.com → misma cuenta Google → alta (país España, moneda EUR). Se crea/vincula una cuenta AdSense para cobros.
2. Crear app → "Android" → al principio "app no publicada aún"; cuando esté en Play, **vincularla a la ficha de Play** para que quede verificada.
3. Crear 3 unidades: banner, interstitial, rewarded. Copiar App ID + los 3 unit IDs a `ads_manager.gd` (build producción).
4. **app-ads.txt:** publicar en la raíz del dominio del desarrollador (el que figure en la ficha de Play) el fichero con la línea que AdMob te da (`google.com, pub-XXXXXXXXXXXXXXXX, DIRECT, f08c47fec0942fa0`). Tienes Hetzner + dominios: trivial, y sin él AdMob limita el servicio de anuncios.
5. Cobros: datos fiscales (autónomo, modelo de la AEAT que corresponda por ingresos de Google Ireland), IBAN, PIN postal de verificación cuando superes el umbral. Pago mensual a partir de 70 €.
6. Consentimiento UE: en AdMob → Privacidad y mensajería → crear mensaje **GDPR (UMP)**; el plugin lo muestra en la primera ejecución. Sin esto, en la UE solo se sirven anuncios limitados o ninguno.

5. CRITERIOS DE "HECHO" (auditoría final antes de release)

- [] 20 niveles jugables seguidos sin error de consola.

- [] Ningún nivel generado imposible (validador de solapamientos OK en 500 semillas: script de test headless).
- [] FPS ≥ 60 en escritorio, ≥ 30 en un Android medio.
- [] Interstitial nunca aparece dos veces en < 90 s; rewarded siempre voluntario.
- [] UMP se muestra en primer arranque (probar con dispositivo con IP UE).
- [] AAB target SDK 36, firmado con keystore release respaldada.
- [] IDs de AdMob reales solo en build de producción; ni un ID de test en el AAB subido.
- [] Política de privacidad publicada y enlazada; app-ads.txt accesible.